Desenvolvimento de Sistemas

Aula 10 - Gerência de Configuração, Infraestrutura, Ambientes (Docker e Containers)

Lorrany B A Marim

SENAI

Introdução à Gerência de Configuração

Organizando a casa antes de programar

- **O que é:** É a disciplina que gerencia as versões de um software, suas dependências e as configurações necessárias para que ele rode perfeitamente em qualquer lugar.
- **O Desafio:** Um software não é feito apenas do código que você digita. Ele depende do sistema operacional, do banco de dados, da versão da linguagem e de bibliotecas externas.
- **DevOps:** No mercado moderno, existe uma cultura chamada DevOps, que aproxima quem cria o sistema (Desenvolvedores) de quem mantém o sistema no ar (Operações), garantindo que a configuração seja padronizada.

“Na minha máqu

café
@cafecomcodig0

tenho uma piada sobre programação, mas só funciona na minha máquina

10:02 AM · 29 de jul de 2020 · Twitter Web App

“Na minha máquina funciona”

Por que os erros acontecem ao trocar de computador?

- **O Cenário:** Você termina o código, testa no seu computador e está perfeito. Quando envia para o servidor ou para o colega, o sistema "quebra".

- **Causa 1 (Falta de Bibliotecas):** O computador de destino não tem a biblioteca de terceiros que o seu sistema exige instalada.
- **Causa 2 (Versão Incorreta):** O computador de destino até tem a biblioteca, mas em uma versão mais antiga ou mais nova que a sua, o que muda a forma como o código reage.

- **Conclusão:** O código é o mesmo, mas o ambiente é diferente. A culpa não é do código, é da falta de padronização.

SENAI

Isolamento de Ambientes (Nível Local)

Criando "bolhas" para o seu projeto

- Para resolver o problema das versões, não instalamos as dependências do projeto no computador inteiro (globalmente).

- **Ambientes Virtuais:** Em linguagens como Python, usamos "ambientes virtuais" (como o virtualenv). É uma pasta exclusiva para o seu projeto que contém todas as dependências isoladas do resto do sistema.

Isolamento de Ambientes (Nível Local)

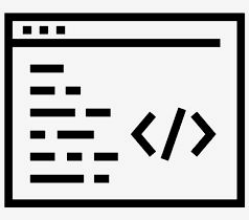
- **Vantagem:** Fazer a atualização de um programa no seu computador não vai quebrar os outros projetos, pois cada um vive no seu próprio ambiente isolado.

- **A Regra:** Os ambientes de Desenvolvimento (sua máquina), Testes e Produção (o servidor final) devem ser o mais idênticos possível.

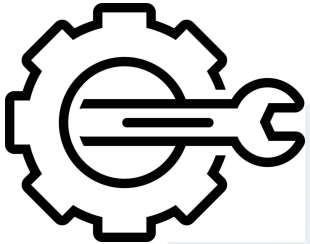
Separando Código de Configuração

O que muda de um ambiente para outro?

- **O Código é estático:** A lógica do programa (os ifs, loops, funções) é exatamente a mesma na sua máquina e no servidor da nuvem.



- **A Configuração é dinâmica:** O que muda são os dados de acesso.
  - **Na sua máquina:** O sistema conecta no banco de dados "localhost".
  - **No servidor:** O sistema conecta no banco de dados da nuvem (ex: AWS, Azure).



- **A Boa Prática:** Nunca escreva dados que mudam de ambiente diretamente dentro do código fonte (hardcoded).

Variáveis de Ambiente e o .env

A forma correta de configurar

- **O que são Variáveis de Ambiente?** São valores guardados no próprio Sistema Operacional e não no código. O programa apenas "pergunta" ao sistema qual é o valor daquela variável no momento da execução.
- **O arquivo .env:** Durante o desenvolvimento, usamos um arquivo de texto simples chamado .env na raiz do projeto para simular essas variáveis de forma prática.
- **Como funciona:** O código lê o arquivo .env, carrega os valores (ex: BANCO_URL=meubanco.com) e usa para se conectar.

Segurança de Credenciais

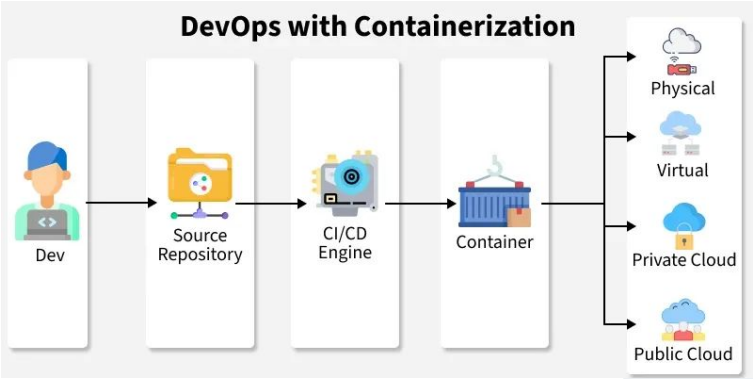
O perigo de vazar senhas

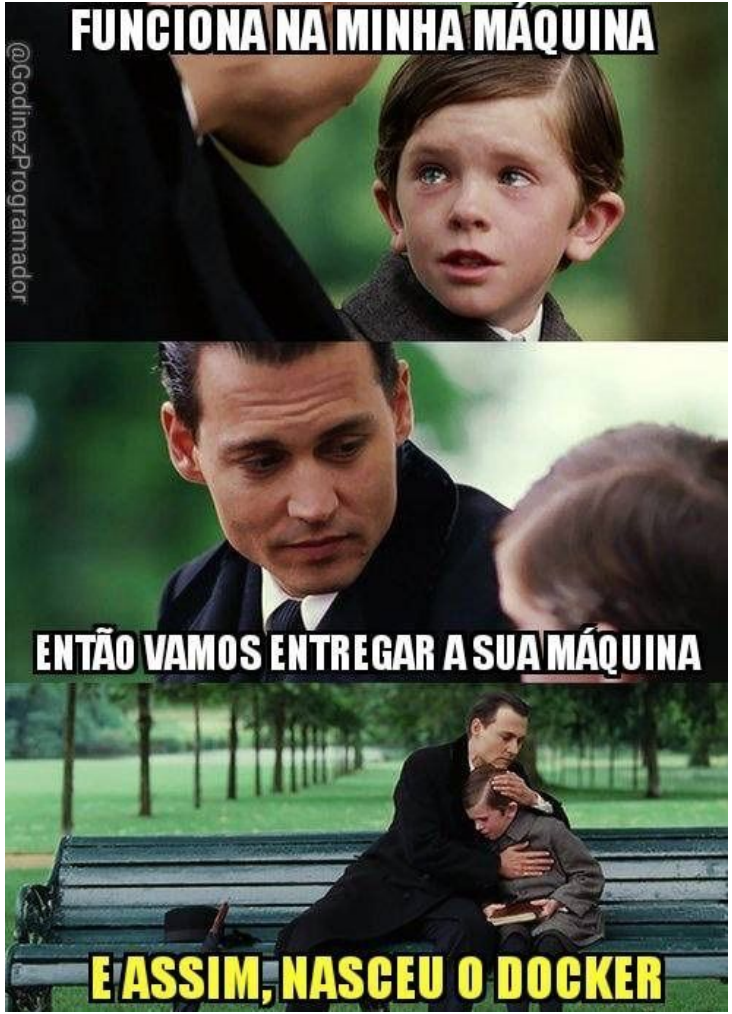
- **O Problema:** Softwares modernos dependem de chaves de APIs e senhas de Banco de Dados.
- **O Risco:** Se você colocar a senha no código e salvar num repositório como o GitHub, qualquer pessoa poderá roubar os dados e invadir a empresa.
- **A Regra de Ouro:** O arquivo .env **NUNCA** deve ser enviado para o repositório de código.
- **Como fazer:** Colocamos o nome do arquivo .env dentro do arquivo .gitignore, para que o Git o ignore e ele fique apenas na sua máquina local.

Evoluindo para Containers

Indo além do isolamento de bibliotecas

- **O Próximo Passo:** Ambientes virtuais (como virtualenv) isolam a linguagem, mas e se precisarmos isolar o sistema operacional inteiro, o banco de dados e as configurações de rede?
- **A Solução:** É aí que entram os **Containers**.
- **Definição:** Um container é uma unidade de software padronizada que empacota o código e TODAS as suas dependências (bibliotecas, ferramentas de sistema, configurações).
- **Portabilidade:** Se o container rodou no seu computador pessoal, ele rodará da mesma forma em qualquer servidor de nuvem.




FUNCIONA NA MINHA MÁQUINA
@GodinezProgramador
ENTÃO VAMOS ENTREGAR A SUA MÁQUINA
E ASSIM, NASCEU O DOCKER

O que é o Docker?

A ferramenta que popularizou os containers

- **Definição:** O Docker é uma plataforma que facilita a criação, a implantação e a execução de aplicações em containers.
- **O "Navio Cargueiro":** Pense no Docker como uma empresa de logística. Não importa se dentro do container há Java ou Python; o "navio" (servidor) sabe como transportá-los de forma padronizada.
- **Simplicidade:** Ele fornece ferramentas para baixar ambientes prontos da internet com um único comando.
- **Docker Hub:** Uma biblioteca pública onde a comunidade compartilha configurações de containers prontas para uso.

Máquina Virtual vs. Container

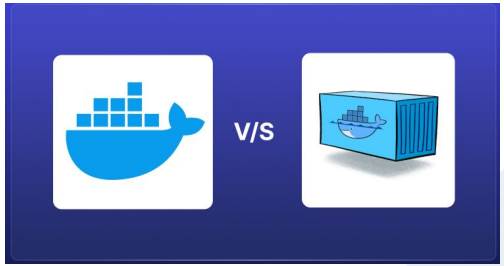
Característica	Máquina Virtual (VM)	Container (Docker)
Virtualização	Virtualiza o Hardware	Virtualiza o Sistema Operacional
Peso	Pesada (Gigas de RAM/Disco)	Leve (Megabytes)
Velocidade	Demora minutos para iniciar	Inicia em segundos
Eficiência	Roda poucas instâncias por servidor	Roda dezenas ou centenas de instâncias

SENAI

Imagem vs. Container

A Planta vs. A Casa

- **Imagem (Image):** É um modelo somente leitura com as instruções para criar um container. É o "molde estático" (como uma **Classe** na programação ou uma **Planta** de construção).
- **Dockerfile:** Arquivo de texto onde o desenvolvedor escreve o passo a passo para construir a imagem.
- **Container (Instância):** É a instância executável de uma Imagem. É a "casa construída" ou o **Objeto** instanciado em memória.
- **Dinâmica:** Você pode subir vários containers idênticos a partir de uma única imagem. Eles são efêmeros: podem ser destruídos e recriados sem afetar a imagem base.



Resumo e Entrega Contínua

Fechando o ciclo DevOps

- **Automação:** Ao ter o ambiente da aplicação como código (Imagens Docker), podemos automatizar todo o processo de entrega.
- **Entrega Contínua (Continuous Delivery):** O Docker permite atingir o Nível 3 de automação, garantindo implantação rápida e confiável [Sabbagh].
- **Checklist do Desenvolvedor Profissional:**
  1. Isole ambientes para evitar o "na minha máquina funciona".
  2. Separe Código de Configuração usando Variáveis de Ambiente.
  3. Proteja senhas com .env e .gitignore.
  4. Use Docker para garantir que o ambiente de teste seja idêntico ao de produção.

Fontes Utilizadas

- *Downey, A. B. "Pense em Python".*
- *Gamma, E. et al. "Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos".*
- *Danjou, J. "Python Levado a Sério".*
- *Valente, M. T. "Engenharia de Software Moderna".*
- *Preece, J. et al. "Design de Interação".*
- *Sabbagh, R. "Scrum: Gestão ágil para projetos de sucesso" (Entrega Contínua).*
- *Documentação do Docker (Oficial)*

Obrigado!

SENAI